

ColumnTransformer#

<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>

Description:

Applies transformers to columns of an array or pandas DataFrame. This estimator allows different columns or column subsets of the input to be transformed separately and the features generated by each transformer will be concatenated to form a single feature space. This is useful for heterogeneous or columnar data, to combine several feature extraction mechanisms or transformations into a single transformer. Read more in the User Guide. Added in version 0.20.

Function Signature

```
class sklearn.compose.ColumnTransformer(transformers, *,
remainder='drop', sparse_threshold=0.3, n_jobs=None,
transformer_weights=None, verbose=False,
verbose_feature_names_out=True,
force_int_remainder_cols='deprecated')[source]#
```

Parameters:

transformerslist of tuples

namestr

transformer{'drop', 'passthrough'} or estimator

```
class sklearn.compose.ColumnTransformer(transformers, *,
remainder='drop', sparse_threshold=0.3, n_jobs=None,
transformer_weights=None, verbose=False,
verbose_feature_names_out=True,
force_int_remainder_cols='deprecated')[source]#
```

Parameters:

transformerslist of tuples

namestr

transformer{'drop', 'passthrough'} or estimator

```
fit(X, y=None, **params)[source]#
```

Parameters:

X{array-like, dataframe} of shape (n_samples, n_features)

yarray-like of shape (n_samples,...), default=None

**paramsdict, default=None

Parameters

`columnsstr, array`

Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where transformer expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data X and can return any of the above. To select multiple columns by name or dtype, you can use `make_column_selector`.

`X{array}`

Input data, of which specified subsets are used to fit the transformers.

`yarray`

Targets for supervised learning.

`**paramsdict, default=None`

Parameters to be passed to the underlying transformers' fit and transform methods. You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`. Added in version 1.4.

`X{array}`

Input data, of which specified subsets are used to fit the transformers.

`yarray`

Targets for supervised learning.

`**paramsdict, default=None`

Parameters to be passed to the underlying transformers' fit and transform methods. You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`. Added in version 1.4.

`X_t{array}`

Horizontally stacked results of transformers. `sum_n_components` is the sum of `n_components` (output dimension) over transformers. If any result is a sparse matrix, everything will be converted to sparse matrices.

`input_featuresarray`

Input features. If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`. If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

`paramsdict`

Parameter names mapped to their values.

`**kwargsdict`

Estimator parameters.

`X{array}`

The data to be transformed by subset.

`**paramsdict, default=None`

Parameters to be passed to the underlying transformers' transform method. You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`. Added in version 1.4.

$X_{t\{array}}$

Horizontally stacked results of transformers. `sum_n_components` is the sum of `n_components` (output dimension) over transformers. If any result is a sparse matrix, everything will be converted to sparse matrices.

Code Examples

```
>>> import numpy as np
>>> from sklearn.compose import ColumnTransformer
>>> from sklearn.preprocessing import Normalizer
>>> ct = ColumnTransformer(
...     [("norm1", Normalizer(norm='l1'), [0, 1]),
...     ("norm2", Normalizer(norm='l1'), slice(2, 4))])
>>> X = np.array([[0., 1., 2., 2.],
...               [1., 1., 0., 1.]])
>>> # Normalizer scales each row of X to unit norm. A separate
scaling
>>> # is applied for the two first and two last elements of each
>>> # row independently.
>>> ct.fit_transform(X)
array([[0. , 1. , 0.5, 0.5],
       [0.5, 0.5, 0. , 1. ]])
```

```

>>> from sklearn.feature_extraction.text import CountVectorizer
>>> from sklearn.preprocessing import MinMaxScaler
>>> import pandas as pd
>>> X = pd.DataFrame({
...     "documents": ["First item", "second one here", "Is this
the last?"],
...     "width": [3, 4, 5],
... })
>>> # "documents" is a string which configures ColumnTransformer
to
>>> # pass the documents column as a 1d array to the
CountVectorizer
>>> ct = ColumnTransformer(
...     [("text_preprocess", CountVectorizer(), "documents"),
...     ("num_preprocess", MinMaxScaler(), ["width"])]
>>> X_trans = ct.fit_transform(X)

```

Notes & Warnings

NOTE Note If you do not access the list of columns for the remainder columns in the `transformers_` fitted attribute, you do not need to set this parameter.

NOTE See also `make_column_transformerConvenience` function for combining the outputs of multiple transformer objects applied to column subsets of the original feature space. `make_column_selectorConvenience` function for selecting columns based on datatype or the columns name with a regex pattern.

Detailed Documentation

Documentation

Applies transformers to columns of an array or pandas DataFrame.

This estimator allows different columns or column subsets of the input to be transformed separately and the features generated by each transformer will be concatenated to form a single feature space. This is useful for heterogeneous or columnar data, to combine several feature extraction mechanisms or transformations into a single transformer.

Read more in the User Guide.

Added in version 0.20.

List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data.

Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using `set_params` and searched in grid search.

Estimator must support fit and transform. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively.

Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where transformer expects X to be a 1d array-like (vector), otherwise a 2d array will be passed to the transformer. A callable is passed the input data X and can return any of the above. To select multiple columns by name or dtype, you can use

`make_column_selector`.

By default, only the specified columns in transformers are transformed and combined in the output, and the non-specified columns are dropped. (default of 'drop'). By specifying `remainder='passthrough'`, all remaining columns that were not specified in transformers, but present in the data passed to fit will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during fit will be excluded from the output of transform. By setting `remainder` to be an estimator, the remaining non-specified columns will use the remainder estimator. The estimator must support fit and transform. Note that using this feature requires that the DataFrame columns input at fit and transform have identical order.

If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use `sparse_threshold=0` to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.

Number of jobs to run in parallel. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details.

Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.

If True, the time elapsed while fitting each transformer will be printed as it is completed.

If True, `ColumnTransformer.get_feature_names_out` will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting `verbose_feature_names_out="{transformer_name}__{feature_name}"`.

If False, `ColumnTransformer.get_feature_names_out` will not prefix any feature names and will error if feature names are not unique.

If Callable[[str, str], str], ColumnTransformer.get_feature_names_out will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name.

If str, it must be a string ready for formatting. The given string will be formatted using two field names: transformer_name and feature_name. e.g. "{feature_name}__{transformer_name}". See str.format method from the standard library for more info.

Added in version 1.0.

Changed in version 1.6: verbose_feature_names_out can be a callable or a string to be formatted.

This parameter has no effect.

If you do not access the list of columns for the remainder columns in the transformers_fitted attribute, you do not need to set this parameter.

Added in version 1.5.

Changed in version 1.7: The default value for force_int_remainder_cols will change from True to False in version 1.7.

Deprecated since version 1.7: force_int_remainder_cols is deprecated and will be removed in 1.9.

The collection of fitted transformers as tuples of (name, fitted_transformer, column). fitted_transformer can be an estimator, or 'drop'; 'passthrough' is replaced with an equivalent FunctionTransformer. In case there were no columns selected, this will be the unfitted transformer. If there are remaining columns, the final element is a tuple of the

form: ('remainder', transformer, remaining_columns) corresponding to the remainder parameter. If there are remaining columns, then `len(transformers_)==len(transformers)+1`, otherwise `len(transformers_)==len(transformers)`.

Added in version 1.7: The format of the remaining columns now attempts to match that of the other transformers: if all columns were provided as column names (str), the remaining columns are stored as column names; if all columns were provided as mask arrays (bool), so are the remaining columns; in all other cases the remaining columns are stored as indices (int).

Access the fitted transformer by name.

Boolean flag indicating whether the output of transform is a sparse matrix or a dense numpy array, which depends on the output of the individual transformers and the `sparse_threshold` keyword.

A dictionary from each transformer name to a slice, where the slice corresponds to indices in the transformed output. This is useful to inspect which transformer is responsible for which transformed feature(s).

Added in version 1.0.

Number of features seen during fit. Only defined if the underlying transformers expose such an attribute when fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Convenience function for combining the outputs of multiple transformer objects applied to column subsets of the original feature space.

Convenience function for selecting columns based on datatype or the columns name with a regex pattern.

The order of the columns in the transformed feature matrix follows the order of how the columns are specified in the transformers list. Columns of the original feature matrix that are not specified are dropped from the resulting transformed feature matrix, unless specified in the `passthrough` keyword. Those columns specified with `passthrough` are added at the right to the output of the transformers.

`ColumnTransformer` can be configured with a transformer that requires a 1d array by setting the column to a string:

For a more detailed example of usage, see [Column Transformer with Mixed Types](#).

Fit all transformers using `X`.

Input data, of which specified subsets are used to fit the transformers.

Targets for supervised learning.

Parameters to be passed to the underlying transformers' fit and transform methods.

You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

Added in version 1.4.

Fit all transformers, transform the data and concatenate results.

Input data, of which specified subsets are used to fit the transformers.

Targets for supervised learning.

Parameters to be passed to the underlying transformers' fit and transform methods.

You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

Added in version 1.4.

Horizontally stacked results of transformers. `sum_n_components` is the sum of `n_components` (output dimension) over transformers. If any result is a sparse matrix, everything will be converted to sparse matrices.

Get output feature names for transformation.

If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`.

If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

Transformed feature names.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

Added in version 1.4.

A `MetadataRouter` encapsulating routing information.

Get parameters for this estimator.

Returns the parameters given in the constructor as well as the estimators contained within the transformers of the ColumnTransformer.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Set the output container when "transform" and "fit_transform" are called.

Calling set_output will set the output of all estimators in transformers and transformers_.

Configure output of transform and fit_transform.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

Valid parameter keys can be listed with get_params(). Note that you can directly set the parameters of the estimators contained in transformers of ColumnTransformer.

Estimator parameters.

Transform X separately by each transformer, concatenate results.

The data to be transformed by subset.

Parameters to be passed to the underlying transformers' transform method.

You can only pass this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

Added in version 1.4.

Horizontally stacked results of transformers. `sum_n_components` is the sum of `n_components` (output dimension) over transformers. If any result is a sparse matrix, everything will be converted to sparse matrices.

Time-related feature engineering

Column Transformer with Heterogeneous Data Sources

Column Transformer with Mixed Types

Partial Dependence and Individual Conditional Expectation Plots

Permutation Importance vs Random Forest Feature Importance (MDI)

Poisson regression and non-normal loss

Tweedie regression on insurance claims

Evaluation of outlier detection estimators

Introducing the `set_output` API

Comparing Target Encoder with Other Encoders

Release Highlights for scikit-learn 1.0

Release Highlights for scikit-learn 1.1

Release Highlights for scikit-learn 1.2

Release Highlights for scikit-learn 1.4

Release Highlights for scikit-learn 1.5

TransformedTargetRegressor

© Copyright 2007 - 2025, scikit-learn developers (BSD License).